From Prompt to Product

Using Claude to ship a feature-rich app — from prompt to application.

alpha.grimify.app → beta.grimify.app → grimify.app

Nathan Healea

- Analyst Programmer II
- Web Application Team — *Information Services*
- 3+ years @ UO
- 10+ years as a Full-Stack Developer & Software Engineer
- Finance · Healthcare · Higher Education

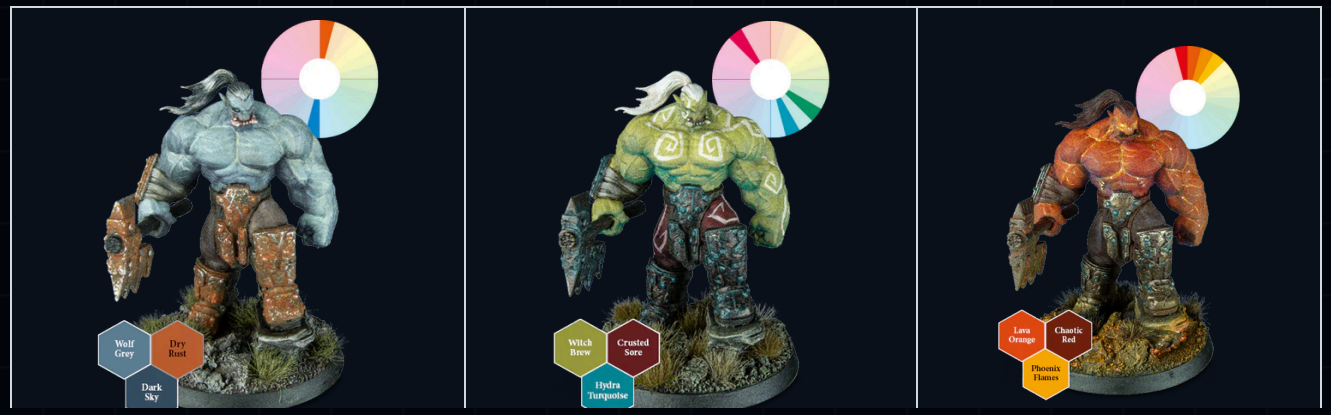
CONTENT WARNING

Weapons · Fantasy Gore · Grimdark Themes

The following slides reference Warhammer miniature painting, which includes fantasy violence, weapons, and grimdark imagery.

"I wish this was interactive."

Inspired by The Army Painter's guide to picking a colour scheme.



Prompt

I'm wanting to create a interactive color wheel for viewing, selecting paints basted on different color theory factors.
The color wheel should have the following features:

- Colors should be placed on the color wheel.
- Darker colors should appear further out, while lighter colors appear further in.
- Colors should be placed on the color wheel based on mathematically based on there hexadecimal values.
- Users should be able to zoom in on the color wheel, when they do so colors should dissipate out showing more gap between repressing there color differences.
- Users should be table to select. color schemes and have those areas emphasized on the color wheel.
  - Complementary Colors
  - Slit Complementary Colors.
  - Analogous Colors.

alpha.grimify.app - Concept

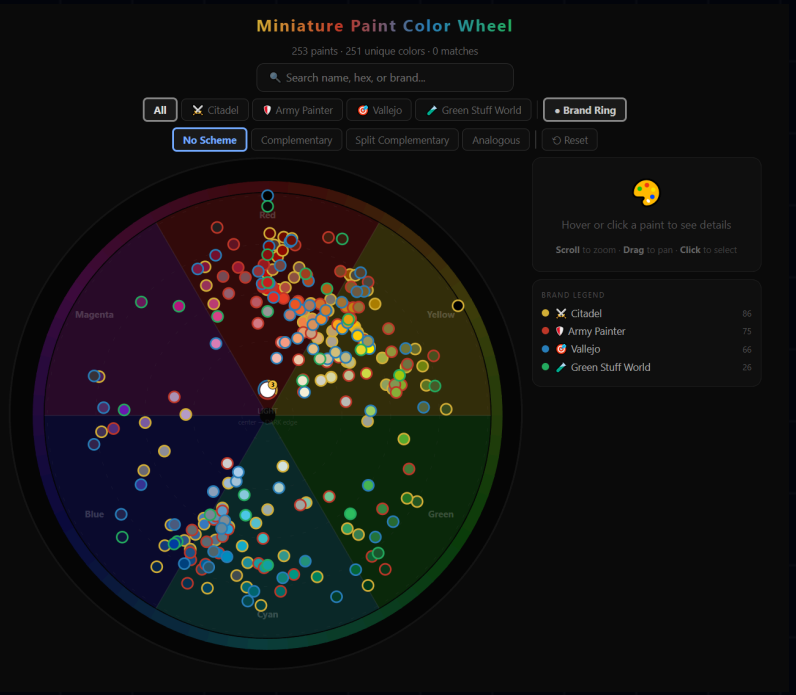
The first version that proved the idea could exist at all.

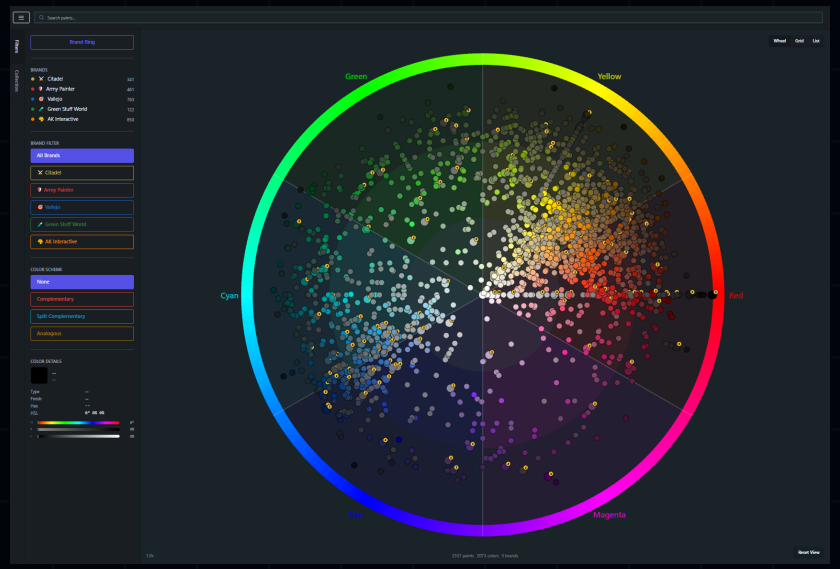
- **Paint Data & Search** — paint/brand/hue data model, seed catalog, search by name, hex, or brand
- **Interactive Color Wheel** — paints rendered by hue and lightness, zoom + pan, paint detail view
- **Color Schemes** — complementary, split-complementary, and analogous color indicators

What Came Next

- **Improved Paint Catalog & Data** — more paints, better color information
- **Personal Collection** — users can catalog the colors they own and identify them on the color wheel
- **Better Filters** — by brand, color scheme, and owned collection

LINK: alpha.grimify.app





beta.grimify.app - Expansion

Once the foundation was real, then the features expanded, problems started to arise.

- **Cross-brand comparison** — color-distance engine, side-by-side UI, similar paints on every paint detail page, substitutes for discontinued paints
- **Color scheme explorer** — complementary, split-complementary, analogous, and triadic scheme generation; schemes surfaced on every paint
- **Collection tracking** — add/remove paints, personal collection dashboard, toast feedback
- **Color wheel, refined** — HSL wheel, Itten segment boundaries, brand rings & halos, filter by brand / collection / owned

Problems

- **User Collection** - Utilized the browser storage, no backend database.
- **User Accounts** - Without a user account, profiles, collections, could not be saved or shared.
- **Data** — expanding the paint catalog meant static files grew large, requiring over 2K paints to load at once

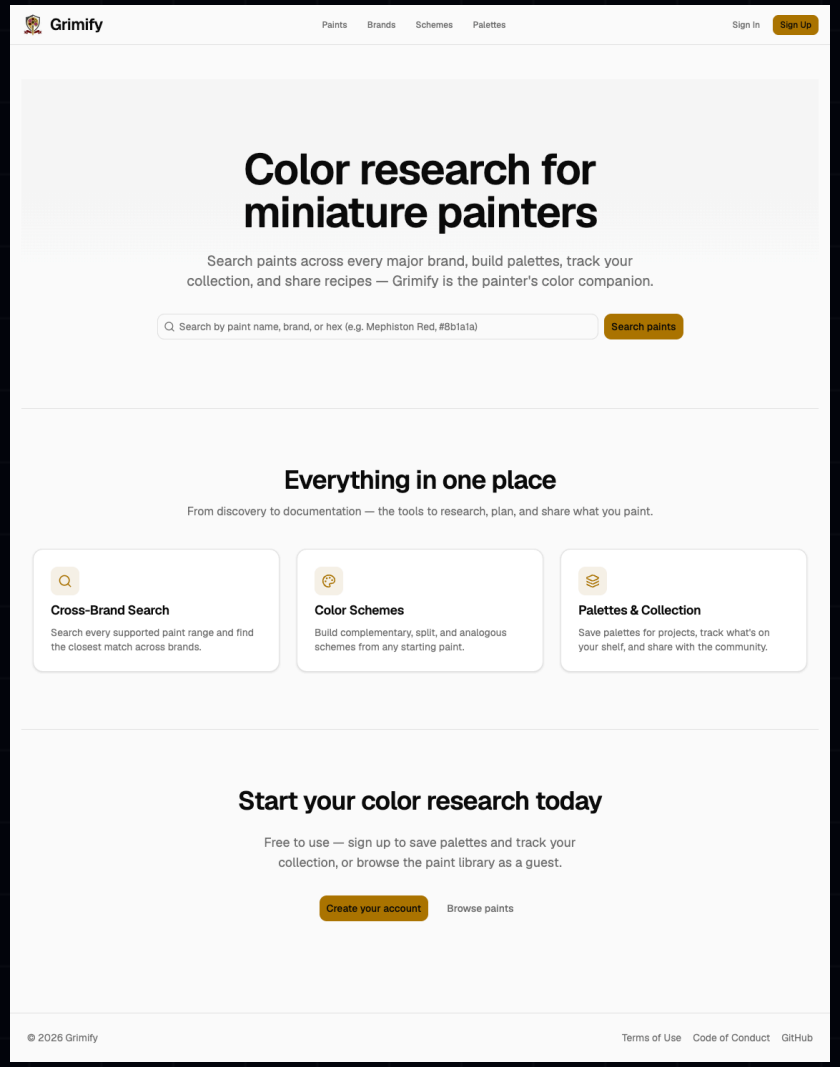
LINK: beta.grimify.app

grimify.app - Polish

The version where Grimify stops being a *tool* and starts being an *application*.

- **Color palettes** — build personal palettes from any source (including the scheme explorer), drag-and-drop reorder, hue-locked HSL swap, markdown descriptions, palette groups, public catalog
- **Painting recipes** — step-by-step recipe builder with sections, techniques, paint ratios, photos, freeform notes
- **Admin & user management** — admin dashboard, role management, account management, collection management
- **Polish everywhere** — toast feedback across auth, collections, palettes, schemes, and admin flows

LINK: grimify.app



What Claude Actually Made Possible

Three deployed environments, dozens of features — built solo, on nights and weekends. None of it works without these two things:

- `CLAUDE.md` **as the project contract** — tech stack, domain-module structure, naming conventions, workflow routing. Claude reads it before every command.
- **A four-command spine** — `/plan` writes the feature doc, `/implement` walks the diff one commit at a time inside a worktree, `/stage` runs build + lint and opens the PR, `/release` cleans up after merge.

The same pattern is now what I reach for at work — different remote, same muscle memory.

LINK: github - claude

Thanks

Questions?